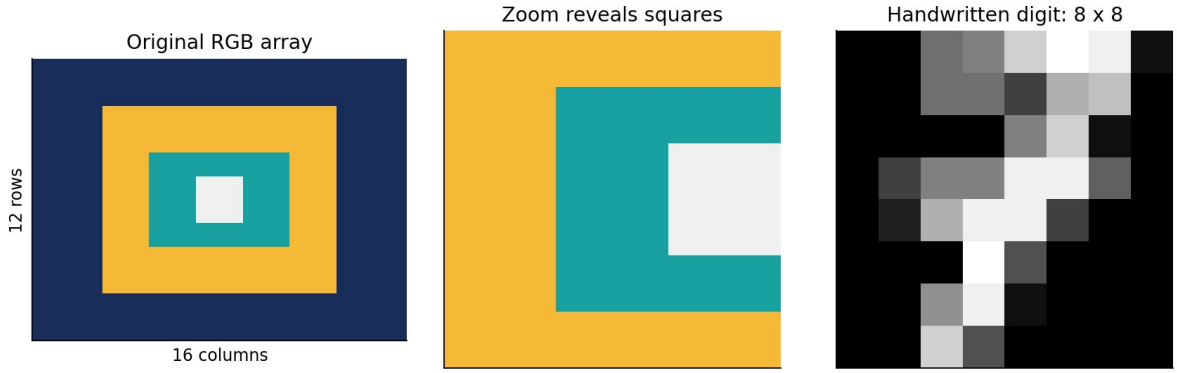
מדעי הבינה המלאכותית

התמחות למידת מכונה ולמידה עמוקה | פרק 6 בתכנית ההתמחות

עיבוד תמונה — מפיקסל בודד ועד לייצוג תמונה במודל AI

מהדורת תלמיד מורחבת · הסברים דידקטיים · נוסחאות וחישובים ידניים · 12 המחשות מקוריות · Python · תרגול עצמאי



איור: התמונה שאנחנו רואים כעצם שלם שמורה במחשב כמערך של ערכי פיקסלים. גם ספרות בכתב יד הן מערכים מספריים.

מיקום בסילבוס: פרק 6 — עיבוד תמונה, 15 שעות, בתכנית ההתמחות בלמידת מכונה ולמידה עמוקה. שיוך זה נבדק מול מפת הסילבוס השמורה במאגר הלימוד. רצף ההוראה והניסויים שלהלן הם עיבוד דידקטי מקורי; אין בידינו ברגע זה אישור שכל תת־נושא הוא דרישה פורמלית נפרדת בבחינה.

מה תדעו לעשות לאחר לימוד הפרק?

להסביר מהו פיקסל וכיצד מיוצגת תמונה כמערך; לזהות גובה, רוחב וערוצי צבע; לחשב ידנית פעולות על פיקסלים; לשנות בהירות וניגודיות בלי גלישה מטווח הערכים; לקרוא היסטוגרמת עוצמות; להבין חיתוך, שינוי גודל וסיכון לאיבוד מידע; לחשב מסנן ממוצע על חלון בגודל 3×3; לזהות את תפקיד סינון הקצוות; לכתוב קוד NumPy ו־Pillow; לבנות מעבדה משחזרת ולתאר כיצד קונבולוציה משמשת בסיס ל־CNN.

מפת דרכים לפרק

בעיה אמיתית → פיקסלים ומטריצות → צבע ותמונה אפורה → קואורדינטות ו־dtype → פעולות על תמונה → היסטוגרמה → חיתוך ושינוי גודל → רעש ומסננים → קונבולוציה → מבוא ל־CNN → הערכה, הטיות ופרויקט גמר.

1. סיפור הפתיחה — כיצד מכונה יכולה לקרוא כתב יד?

דודו בונה יישומון לתלמידים: מצלמים מספר שנכתב על דף, והיישומון מזהה את הספרה. לנו קל להבין שתמונה מציגה את הספרה 7, אך למחשב אין מושג טבעי של ״ספרה״ או ״קו״. הוא מקבל אוסף ערכים מספריים. כדי להגיע למערכת שמזהה כתב יד, עלינו להבין קודם מה נשמר בקובץ תמונה, כיצד משנים את הערכים וכיצד אפשר להפיק מהם דפוסים משמעותיים.

נניח שהמצלמה צילמה מספר בתאורה חלשה. התלמיד רואה ספרה כהה, אבל מכונה עלולה לקבל ערכים שההבדל בינם לבין הרקע קטן. לפני שנבנה מסווג, נצטרך לדעת לבדוק בהירות, להתאים גודל, להסיר רעש ולשמור על המידע החשוב. עיבוד תמונה אינו ״קישוט״: הוא עשוי לשנות את הקלט ואת איכות החיזוי של המודל.

שימו לב: שלב עיבוד התמונה יכול להתבצע בעזרת כללים קבועים שהמתכנת הגדיר, גם בלי למידת מכונה. רשת CNN תלמד בהמשך מסננים מתאימים מתוך נתוני האימון; ההבחנה הזאת היא אחת ממטרות הפרק.

בדיקת הבנה: אם נבהיר תמונה כהה, האם נוסיף מידע חדש לתמונה או נשנה את אופן הייצוג של המידע שכבר קיים?

2. מהו פיקסל ומהי רזולוציה?

פיקסל הוא יחידת מדידה במבנה של תמונה דיגיטלית. בתמונה אפורה, כל תא במטריצה מחזיק מספר המתאר עוצמת אור או בהירות. בתמונה צבעונית, לכל מיקום קיימים בדרך כלל כמה ערוצים — למשל אדום, ירוק וכחול. כאשר מציגים את הערכים יחד על מסך, העין שלנו רואה תמונה רציפה, אך במבט מוגדל אפשר להבחין בריבועים הקטנים.

רזולוציה מתארת את מספר הפיקסלים: גובה H כפול רוחב W. תמונה ברוחב 16 פיקסלים ובגובה 12 פיקסלים מכילה 192 מיקומי פיקסלים. אם זו תמונת RGB בת שלושה ערוצים, אפשר לייצג את הצבעים באמצעות 576 ערכי ערוץ. ככל שיש יותר פיקסלים אפשר לשמור פרטים מרחביים קטנים יותר, אך נדרש גם יותר זיכרון וחישוב.

$$N_{pixels} = H \times W \quad | \quad N_{RGB_values} = H \times W \times 3$$

נתון	גובה H	רוחב W	מספר פיקסלים	ערכי RGB
תמונה קטנה	12	16	192	576
תמונה רגילה	32	32	1,024	3,072
קלט גדול	224	224	50,176	150,528

הדוגמה האחרונה גם מבהירה למה קלט תמונה עשוי להיות יקר יותר מקלט טבלאי: במקום כמה מאפיינים, המודל מקבל מאות אלפי מספרים. המספרים אינם בלתי תלויים: פיקסלים סמוכים נוטים להשתייך לאותו קו, קצה או מרקם.

בדיקת הבנה: אם נכפיל את הגובה ואת הרוחב פי שניים, פי כמה יגדל מספר הפיקסלים?

3. תמונה אפורה היא מטריצה

נתחיל מתמונה בגווני אפור. נהוג להשתמש ב־uint8, סוג נתונים של מספר שלם בלתי שלילי שמיוצג בשמונה סיביות. לכן כל ערך יכול לנוע בין 0 ל־255: 0 הוא שחור, 255 הוא לבן, וערכי הביניים הם גוונים שונים של אפור. לא כל תמונה נשמרת כך: קיימות תמונות עם עומק צבע אחר, עם ערכים עשרוניים ועם קידוד אחר. אנו משתמשים ב־uint8 כדי להכיר את העיקרון בצורה ברורה.

עמודה 0	עמודה 1	עמודה 2	עמודה 3
0	0	0	0
0	80	160	0
0	160	255	0
0	0	0	0

כדי להציג את התמונה, הספרייה מתרגמת כל מספר לגוון מתאים. שימו לב שהערכים שבתוך המטריצה הם הנתונים עצמם, בעוד שצבעי התרשים הם דרך ההצגה שלהם. שינוי מפת הצבעים בתצוגה אינו בהכרח משנה את המטריצה.

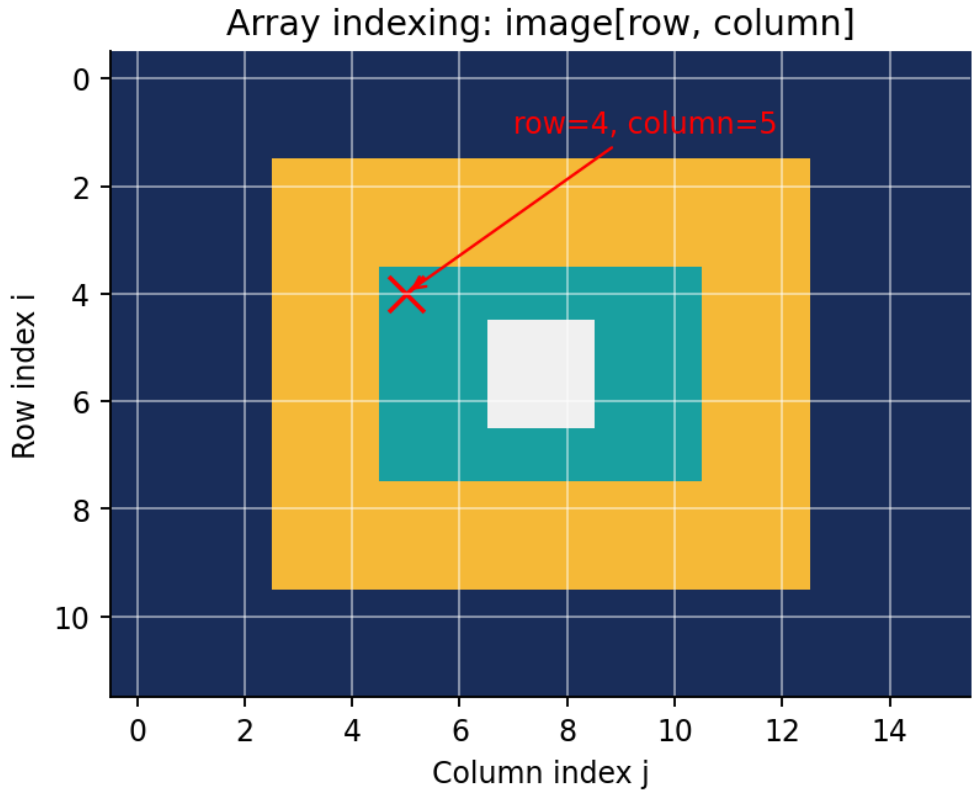
```
import numpy as np
import matplotlib.pyplot as plt
pixels = np.array([[0, 0, 0, 0],
                   [0, 80, 160, 0],
                   [0, 160, 255, 0],
```

```
                  [0, 0, 0, 0]], dtype=np.uint8)
print(pixels.shape)   # (4, 4)
plt.imshow(pixels, cmap="gray", vmin=0, vmax=255)
plt.show()
```

חשוב לזכור: מטריצה היא ייצוג של התמונה, אך אינה יודעת בעצמה מה מופיע בה. פירוש הסימנים שבתמונה הוא כבר משימה ברמה גבוהה יותר.

4. היכן נמצא פיקסל? שורות, עמודות ואינדקסים

בעבודה עם NumPy מציינים קודם שורה ואחר כך עמודה: image[row, column]. השורה הראשונה היא שורה 0; העמודה הראשונה היא עמודה 0. ציר השורות מתקדם בדרך כלל מלמעלה למטה בתמונה, וציר העמודות משמאל לימין. הדבר שונה במידת מה מהאינטואיציה של מערכות צירים במתמטיקה, שבהן ציר y נוטה לעלות כלפי מעלה.



איור: אינדקס של פיקסל: תחילה מספר שורה ואז מספר עמודה. נקודת (4,5) כאן מייצגת שורה 4 ועמודה 5.

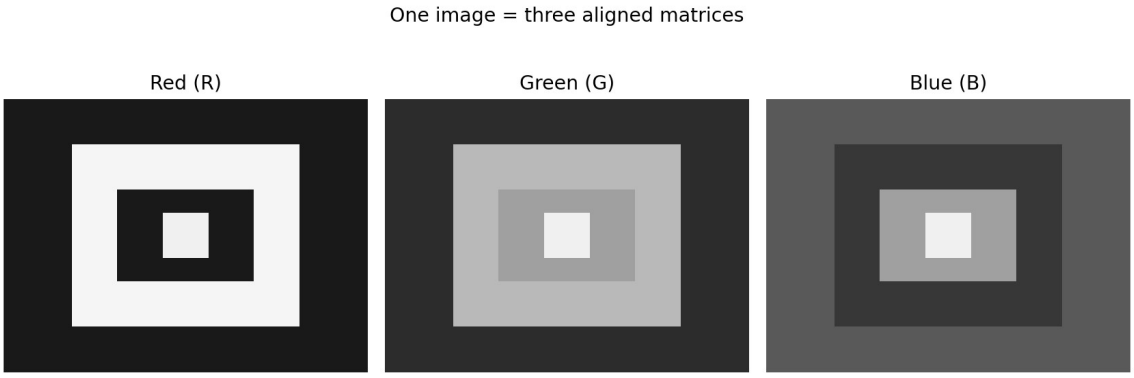
```
rgb = np.zeros((12, 16, 3), dtype=np.uint8)
rgb[:] = [25, 45, 90]
rgb[2:10, 3:13] = [245, 185, 55]
rgb[4:8, 5:11] = [25, 160, 160]
pixel = rgb[4, 5]       # three channel values
red_value = rgb[4, 5, 0]
print(rgb.shape)         # (12, 16, 3)
```

במרבית הפעולות נרצה לשנות אזור שלם ולא תא אחד. בעזרת חיתוך מערך אפשר לבחור טווח שורות וטווח עמודות. הביטוי image[2:8, 3:12] כולל שורות 2 עד 7 ועמודות 3 עד 11, מפני שהגבול העליון אינו נכלל בחיתוך.

בדיקת הבנה: בתמונה שממדיה 12×16, האם image[12,0] חוקי? ומהו אינדקס השורה האחרון?

5. תמונה צבעונית: שלושה ערוצי RGB

בייצוג RGB שומרים לכל פיקסל שלושה ערכים: עוצמת האדום (Red), הירוק (Green) והכחול (Blue). שלושת הערכים יוצרים צבע אחד על המסך. לדוגמה, RGB=(255,0,0) ייתן אדום מלא, RGB=(0,255,0) ירוק מלא, ו־RGB=(255,255,255) לבן. מערך RGB סטנדרטי ב־NumPy בנוי בגודל H×W×3.



איור: כל ערוץ צבע הוא מטריצה דו־ממדית נפרדת. שילוב שלושת הערוצים יוצר את התמונה הצבעונית.

חשוב להבחין בין גודל התמונה לבין מספר הערכים בכל פיקסל. תמונה בגודל 12×16 RGB עדיין מכילה 192 מיקומי פיקסלים, אך כל מיקום כולל שלושה מספרים. כאשר מנתחים משקל קובץ צריך גם לזכור שקובצי PNG/JPEG משתמשים בדחיסה, ולכן מספר הערכים בזיכרון אינו בהכרח שווה לגודל הקובץ על הדיסק.

צבע	R	G	B
אדום	255	0	0
ירוק	0	255	0
כחול	0	0	255
צהוב	255	255	0
אפור בינוני	128	128	128

```
from PIL import Image
color_image = Image.fromarray(rgb)
print(np.asarray(color_image).shape)
red_channel = np.asarray(color_image)[:, :, 0]
plt.imshow(red_channel, cmap="gray", vmin=0, vmax=255)
plt.show()
```

חשוב לזכור: ב־Pillow וב־Matplotlib הצבעים מטופלים לרוב כסדר RGB. פונקציות מסוימות ב־OpenCV משתמשות ב־BGR; בעת מעבר בין ספריות חובה לבדוק את סדר הערוצים.

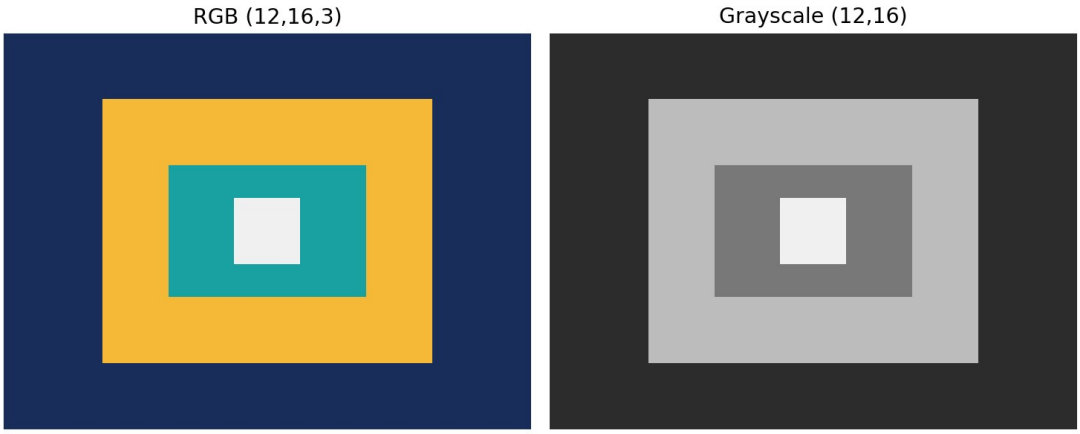
6. המרה לגווני אפור: מדוע לא עושים רק ממוצע פשוט?

לעיתים נרצה לצמצם תמונה צבעונית לערוץ אחד, למשל כדי להתמקד בקווי מתאר של ספרה. אפשר היה לחשב ממוצע רגיל של R,G,B, אך העין האנושית אינה רגישה לכל שלושת הצבעים באותה מידה. לכן מקובל לחשב קירוב לבהירות בעזרת משקולות שונות לערוצים. זו אחת משיטות ההמרה המקובלות, ואינה כל השיטות האפשריות.

$$\text{Gray} \approx 0.299R + 0.587G + 0.114B$$

נחשב ידנית פיקסל שערכיו R=200, G=100, B=50. התרומה של האדום היא 59.8, של הירוק 58.7 ושל הכחול 5.7. הסכום הוא 124.2, ולכן אם נדרוש ערך שלם נקבל בערך 124. לעומת זאת, ממוצע פשוט של שלושת הערכים הוא 116.67. ההבדל ממחיש שהמרה לגווני אפור היא כלל חישוב ולא היעלמות קסומה של הצבע.

0.299×200 + 0.587×100 + 0.114×50 = 124.2



איור: אותה תמונה ב־*RGB* ובגווני אפור: הממדים עוברים מ־$H \times W \times 3$ ל־$H \times W$.

```
rgb_float = rgb.astype(np.float32)
gray = 0.299 * rgb_float[:, :, 0]      + 0.587 * rgb_float[:, :, 1]      + 0.114 *
rgb_float[:, :, 2]
gray_u8 = np.clip(np.rint(gray), 0, 255).astype(np.uint8)
print(gray_u8.shape)
```

בדיקת הבנה: מדוע המרה לגווני אפור עלולה לפגוע במסווג שמבחין בין תפוחים אדומים לירוקים?

7. בהירות וניגודיות — לשנות פיקסלים בצורה מבוקרת

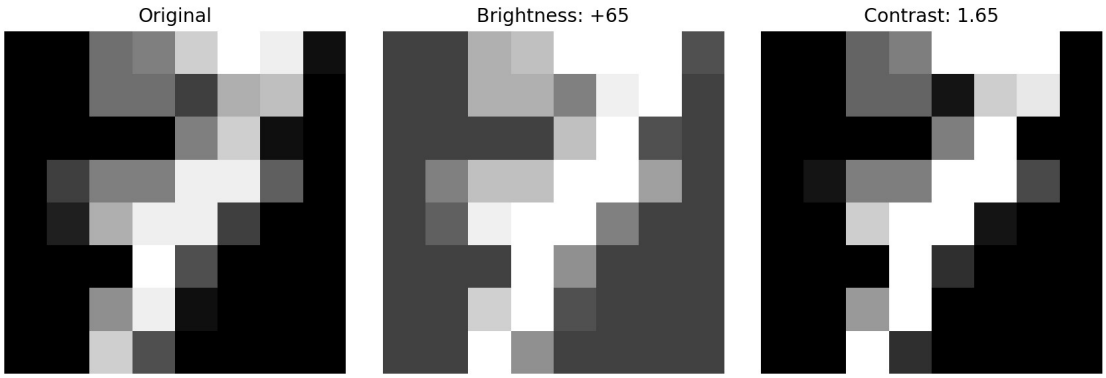
בהירות מתארת את הרמה הכללית של ערכי העוצמה; ניגודיות מתארת את ההבדל בין אזורים בהירים לכהים. אם תמונה חשוכה מדי אפשר לנסות להוסיף לכל ערך קבוע חיובי. אבל אם הפיקסל היה כבר 240 ונוסיף לו 40, התוצאה 280 חורגת מתחום 0–255. אין לשמור אותה ישירות כ־uint8, מפני שפעולות על מספרים בתחום מוגבל עלולות לגרום לגלישה או לליפוף ערכים במקום לרוויה לבנה. לכן מחשבים תחילה בסוג מספרי רחב וחותכים לטווח התקין.

brightness: I_new = clip(I_old + β, 0, 255)

פיקסל מקורי	תוספת β	תוצאה לפני clip	תוצאה סופית
60	40	100	100
220	40	260	255
0	40	40	40

ניגודיות אפשר לשנות באמצעות מתיחה של המרחק מהערך 128: ככל שמקדם α גדול מ־1, ערכים שמעל 128 מתרחקים כלפי מעלה וערכים שמתחתיו מתרחקים כלפי מטה. כאשר α בין 0 ל־1 ההבדלים נחלשים. נצטרך תמיד לחתוך את התוצאות לתחום החוקי. הגדלת ניגודיות עשויה להבליט קווים, אך עלולה גם לאבד פרטים אם אזורים רבים מגיעים ל־0 או ל־255.

contrast: I_new = clip(α(I_old − 128) + 128, 0, 255)



איור: אותה ספרה מוצגת עם בהירות וניגודיות שונים; התמונה נשמרת כמערך, אך הערכים משתנים.

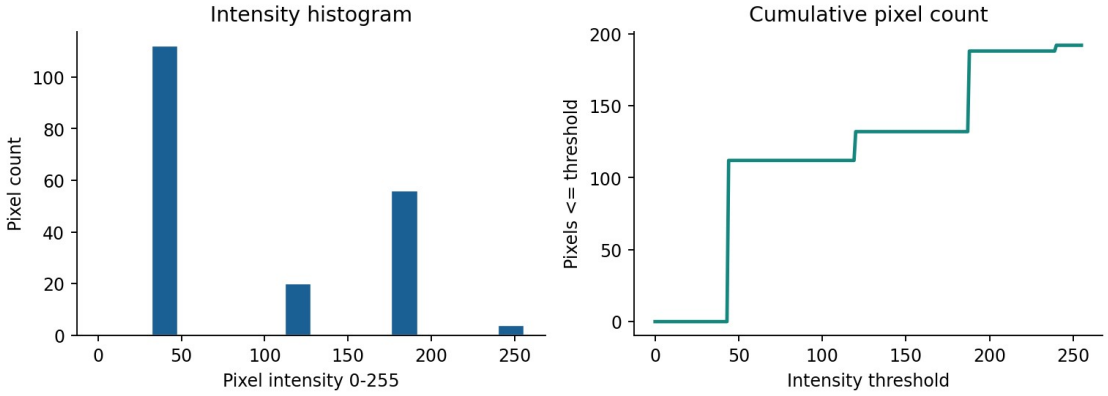
```
from sklearn.datasets import load_digits
digit8 = np.rint(load_digits().images[7] * 255 / 16).astype(np.uint8)
img_float = digit8.astype(np.float32)
brighter = np.clip(img_float + 65, 0, 255).astype(np.uint8)
stronger = np.clip(1.65 * (img_float - 128) + 128,
                   0, 255).astype(np.uint8)
```

חשוב לזכור: חשוב לשמור עותק של התמונה המקורית, כדי שאפשר יהיה לשחזר ולבדוק את השפעת כל שינוי בנפרד.

8. היסטוגרמת בהירות — לקרוא את התפלגות הפיקסלים

היסטוגרמה של תמונת אפור סופרת כמה פיקסלים נמצאים בכל טווח עוצמות. כאשר רוב הפיקסלים נמצאים בטווח נמוך, ייתכן שהתמונה כהה. כאשר הם מרוכזים בטווח גבוה, ייתכן שהיא בהירה. עם זאת, המשמעות תלויה בתוכן התמונה: תמונת לילה אמורה להכיל פיקסלים כהים רבים ואין בכך בהכרח תקלה.

נניח שלתמונה קטנה שישה פיקסלים: 0,0,50,50,50,255. ערך 0 מופיע פעמיים, 50 שלוש פעמים ו־255 פעם אחת. היסטוגרמה היא סיכום של עוצמות ולא של מיקומים. שתי תמונות שונות עשויות לקבל בדיוק אותה היסטוגרמה אם סידרנו מחדש את הפיקסלים, אף שהן מציגות צורות אחרות לחלוטין.



איור: היסטוגרמה סופרת עוצמות, אך אינה שומרת מידע על מיקום הפיקסלים.

```
pixel_values = gray_u8.ravel()
plt.hist(pixel_values, bins=np.arange(0, 257, 16),
         edgecolor="white")
plt.xlabel("Pixel intensity")
plt.ylabel("Count")
plt.show()
```

בדיקת הבנה: האם היסטוגרמה זהה של שתי תמונות מוכיחה שהתמונות זהות? הציגו דוגמה נגדית.

9. חיתוך תמונה ושינוי גודל — מדוע סדר הפעולות חשוב?

חיתוך (Crop) בוחר רק אזור מסוים מתוך התמונה. בתמונה עם מספר ועם רקע גדול מסביב, חיתוך יכול להסיר מידע שאינו מועיל ולהשאיר את הספרה במרכז. אבל חיתוך אגרסיבי עלול לחתוך גם חלק מהספרה. שינוי גודל (Resize) מתאים את ממדי התמונה לגודל הקלט הנדרש במודל, למשל 28×28. פעולה זו אינה מוסיפה מחדש פרטים שנעלמו בצילום המקורי.

שינוי הגודל מחייב כלל להערכת הערכים החדשים. באינטרפולציה הקרובה ביותר בוחרים ערך של פיקסל סמוך; בשיטה ביליניארית מחשבים שילוב משוקלל של כמה שכנים. לכן אותה תמונה יכולה להיראות משוננת או חלקה יותר לפי שיטת שינוי הגודל. חשוב לשמור על יחס רוחב־גובה, או להחליט במודע אם מוסיפים שוליים (Padding), כדי שלא לעוות את האובייקט הנלמד.



איור: אותו מערך קטן לאחר הגדלה בשתי שיטות: הפרטים החדשים הם אומדנים ולא מידע שנמדד במצלמה.

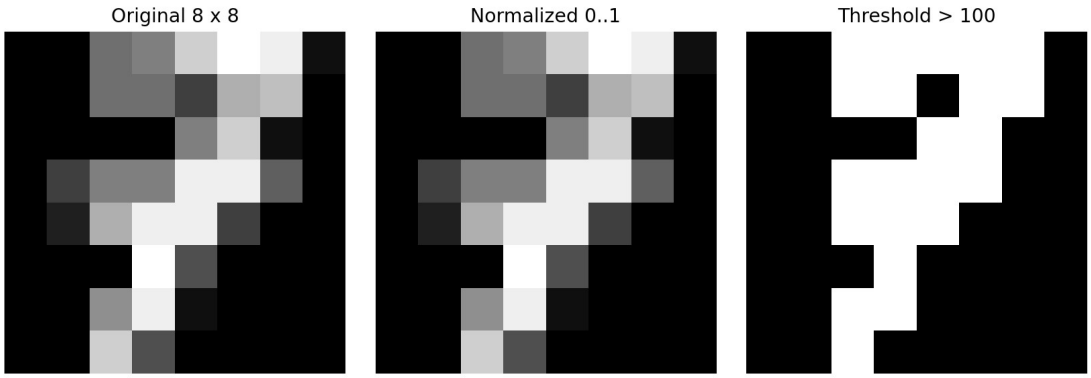
```
from PIL import Image
im = Image.fromarray(rgb)
crop = im.crop((3, 2, 13, 10))  # left, top, right, bottom
resized = im.resize((80, 60), Image.Resampling.BILINEAR)
print(np.array(crop).shape)
print(np.array(resized).shape)
```

חשוב לזכור: שימו לב להבדל: בחיתוך NumPy מסדרים [שורות, עמודות], ואילו Pillow.crop מקבל (left, top, right, bottom).

10. נרמול עוצמות — מ־0 עד 255 לטווח 0 עד 1

מודלים מסוימים נוחים יותר לאימון כאשר ערכי הקלט נמצאים בטווח מוגבל ומוכר. שינוי פשוט הוא חלוקת כל ערכי הפיקסלים ב־255. הפעולה אינה משנה את הסדר היחסי של הערכים ואף אינה משפרת אוטומטית את איכות התמונה; היא רק משנה את יחידות המדידה. פיקסל שהיה 128 יהפוך ל־0.50196 בקירוב, ופיקסל שהיה 255 יהפוך ל־1.

I_normalized = I_uint8 / 255.0



איור: הדגמה של אותם פיקסלים כערכי 0–255, כערכים עשרוניים בטווח 0–1 וכסף בינארי.

סף בינארי (Thresholding) הוא פעולה אחרת לחלוטין: למשל לקבוע שכל ערך מעל 100 יהפוך ל־1, וכל ערך אחר ל־0. הסף יכול לסייע בהפרדת ספרה מרקע, אבל הוא מוחק את רוב המידע על רמות הבהירות. לכן אין לבלבל בין נרמול לבין בינאריזציה.

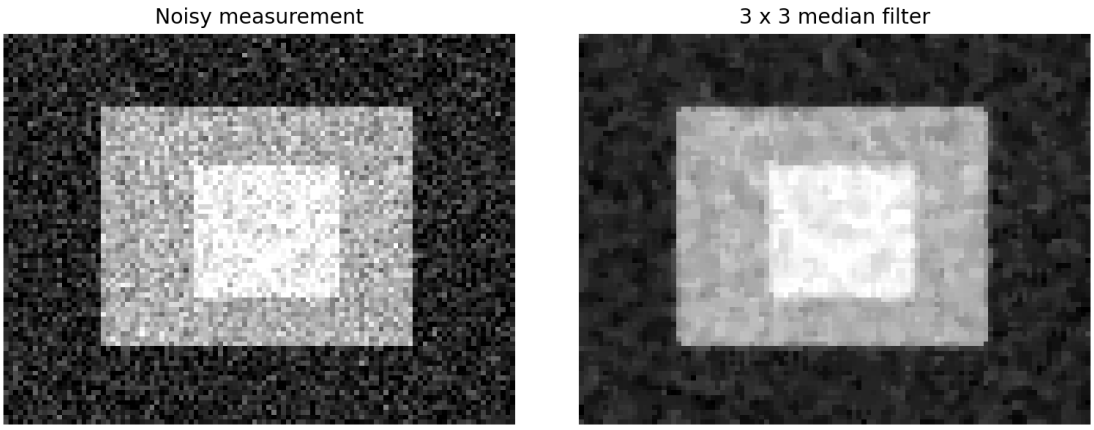
```
normalized = digit8.astype(np.float32) / 255.0
binary = (digit8 > 100).astype(np.uint8)
print(normalized.min(), normalized.max())
print(np.unique(binary))
```

חשוב לזכור: חלוקה ב־255 היא שינוי קבוע שאינו לומד פרמטר מנתוני האימון. לעומת זאת, תקנון לפי ממוצע וסטיית תקן מחייב התאמת הפרמטרים רק על Train, כדי למנוע דליפת מידע.

11. רעש — מה הוא עושה לתמונה?

בצילום אמיתי מופיעים לפעמים הפרעות: הבדלי תאורה, גרעיניות, נקודות בודדות או פיקסלים שאינם מייצגים היטב את המציאות. נוח לחשוב על רעש כהפרעה שמתווספת לאות הרצוי, אך לא כל שינוי קטן הוא רעש — לעיתים זהו פרט חשוב, כגון נקודה מעל אות או קצה של ספרה.

כדי להמחיש זאת נייצר תמונה פשוטה ונוסיף רעש אקראי. אפשר לנסות להקטין רעש באמצעות חציון מקומי: לוקחים את ערכי הסביבה, ממיינים ובוחרים את הערך האמצעי. מסנן חציון מועיל במיוחד לרעש של נקודות חריגות, אבל אינו מתאים בהכרח לכל סוג רעש. חשוב להשוות לתמונה המקורית ולוודא שלא נמחקו פרטים רלוונטיים.



איור: מסנן חציון מטפל ברעש מקומי, אך לעיתים גם מסיר פרטים קטנים.

בדיקת הבנה: אם מסנן מסיר נקודה מיותרת אך גם מסיר נקודה השייכת לאות ״ו״ — האם שיפרנו את התמונה? לפי איזו מטרה צריך להחליט?

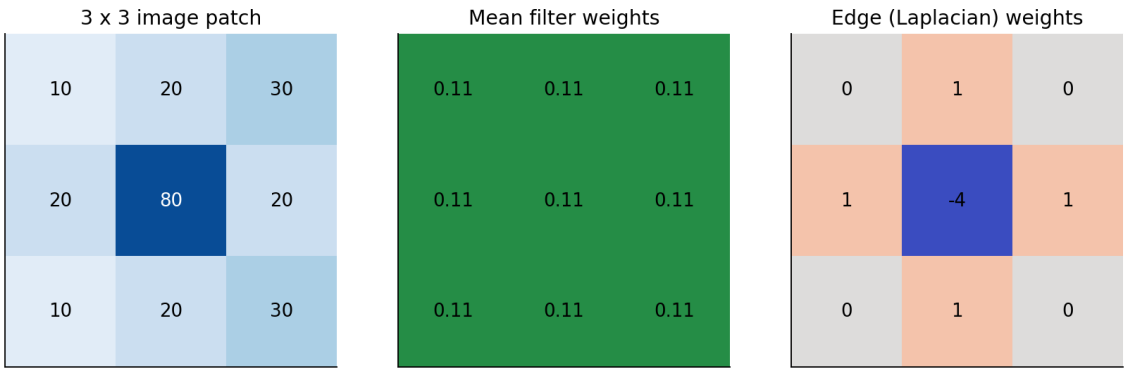
12. חלון שכנים ומסנן ממוצע — חישוב ידני

כעת נגיע לרעיון מרכזי בעיבוד תמונה: פיקסל אינו בהכרח מנותח לבדו. לעיתים נרצה לחקור את סביבתו הקרובה בעזרת חלון קטן, למשל שלוש שורות ושלוש עמודות. מסנן ממוצע מחליף את ערך הפיקסל המרכזי בממוצע של תשעת הערכים שבחלון. התוצאה נוטה להיות חלקה יותר, כי הבדלים חדים בין שכנים מצטמצמים.

10	20	30
20	80	20
10	20	30

בחלון שלפנינו סכום הערכים הוא 240. מחלקים בתשע מפני שיש בו תשעה תאים, ומקבלים 26.67 בקירוב. כלומר הפיקסל המרכזי שערכו 80 מקבל ערך חדש הקרוב ל־27. מה הרווחנו? שיטחנו שינוי מקומי חד. מה איבדנו? ייתכן שערך 80 היה סימן חשוב ולא רעש.

mean(3×3 patch) = 240 / 9 ≈ 26.67



איור: חלון 3×3, מסנן ממוצע ומשקולות מסנן להדגשת שינויים.

מסנן ממוצע הוא מקרה פשוט של פעולה משוקללת. אם לכל תא במשקל יש ערך 1/9, מכפילים כל פיקסל במשקל המתאים ומחברים. אפשר לבחור משקולות אחרות כדי להדגיש קצוות, לטשטש או לבצע פעולות נוספות.

```
patch = np.array([[10,20,30],
                  [20,80,20],
                  [10,20,30]], dtype=float)
weights = np.ones((3,3)) / 9
result = np.sum(patch * weights)
print(round(result, 2))   # 26.67
```

13. קונבולוציה ומסננים: הרעיון המתמטי

קונבולוציה היא פעולה שבה מעבירים מסנן קטן על פני מערך ומחשבים בכל מיקום סכום של מכפלות בין ערכי הסביבה למשקולות המסנן. כדי להבין את המנגנון, דמיינו חלון של 3×3 שמונח על תשעה פיקסלים. בכל צעד מכפילים תא מול תא, מחברים את תשע המכפלות ורושמים ערך בפלט; לאחר מכן מזיזים את החלון למיקום הבא.

$$S(i,j) = \Sigma_a \Sigma_b \; I(i+a, j+b) \cdot K(a,b)$$

הנוסחה שלפנינו היא למעשה פעולת Cross-Correlation, כפי שממומשת בדרך כלל בשכבות CNN. בקונבולוציה המתמטית הקלאסית יש גם היפוך של גרעין הסינון. להדגמת מסנן ממוצע סימטרי שתי הפעולות נותנות את אותה תוצאה; במסננים לא־סימטריים חשוב להבחין ביניהן. בפרק CNN נעסוק גם במשקולות שנלמדות מתוך נתונים.

מה נעשה בשוליים? חלון של 3×3 אינו יכול להיות ממורכז בפינה בלי לצאת מגבולות התמונה. אחת האפשרויות היא לחשב רק במיקומים שיש להם חלון מלא (Valid), ואז גובה ורוחב הפלט קטנים בשניים. אפשרות אחרת היא להוסיף שכבת שוליים מלאכותית (Padding), למשל אפסים, ולשמור על הגודל. הבחירה יכולה להשפיע על התוצאות ליד גבולות התמונה.

$$\text{valid 3×3 filter: } H_{out} = H - 2 \quad ; \quad W_{out} = W - 2$$

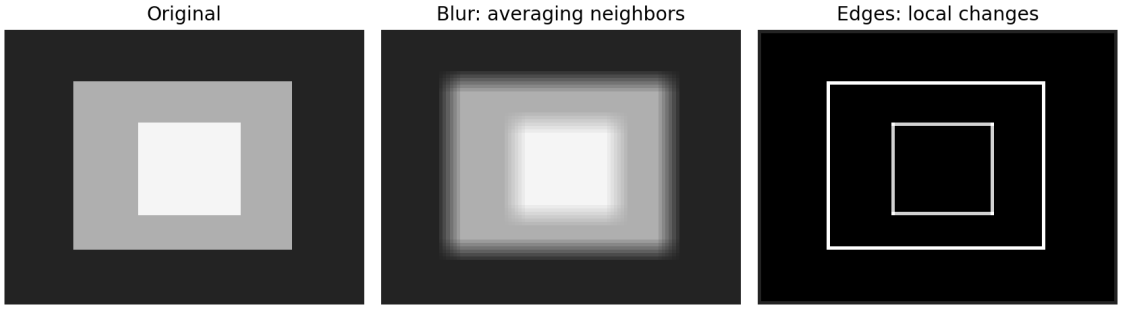
חשוב לזכור: כאשר stride = 1 וללא padding. בהמשך, עבור stride גדול יותר או padding אחר, נוסחת גודל הפלט משתנה.

14. זיהוי קצוות — מה אפשר ללמוד מהפרשים בין שכנים?

קצה בתמונה הוא אזור שבו ערכי הפיקסלים משתנים בחדות במרחב. למשל, במעבר מקיר כהה לחלון מואר נראה הבדל משמעותי בין פיקסלים סמוכים. מסננים מסוימים נועדו להבליט שינויים כאלה. קצה אינו ״שם של עצם״; הוא רק רמז מקומי שעשוי לסייע לזיהוי צורות.

נבחן מסנן סימטרי המדגיש הבדלים בין המרכז לארבעת שכניו הישירים. מקדמי השכנים הם 1, מקדם המרכז הוא מינוס 4, ובפינות מקדם 0. עבור חלון שבמרכזו 80 ובארבעת השכנים הישירים 20 נקבל 80 פחות 320, כלומר מינוס 240. לעיתים מציגים את גודל התגובה או ממירים אותה לטווח מותר לשם תצוגה. הסימן והערך המקורי עשויים להיות חשובים בחישוב.

$$\text{response} = 20+20+20+20 - 4\times 80 = -240$$



איור: טשטוש מפחית פרטים חדים; מסנן קצוות מדגיש שינויים מקומיים בעוצמה.

ב־CNN המסננים יכולים ללמוד להבליט דפוסים מתוך תמונות אימון, במקום שהמתכנת יבחר מראש את כל המשקולות. אולם תהליך הסכימה המקומית של ערכים כפול משקולות הוא אותו רעיון מרכזי.

15. מ־Pillow ו־NumPy אל ייצוג המתאים ללמידת מכונה

כאשר נטען תמונה ל־NumPy נקבל מערך מספרי. בשלב הבא יכול להיות שנרצה להפוך אותו לווקטור אחד ארוך, למשל 8×8 לאוסף של 64 תכונות. כך אפשר להפעיל מסווג רגיל כגון KNN. החיסרון הוא שהמיקום המרחבי פחות מפורש: פיקסלים צמודים בשורות שונות עלולים להתרחק במערך השטוח מבחינה אינדקסית, וקשרים מקומיים אינם נשמרים כשכבה נפרדת של תהליך הלמידה.

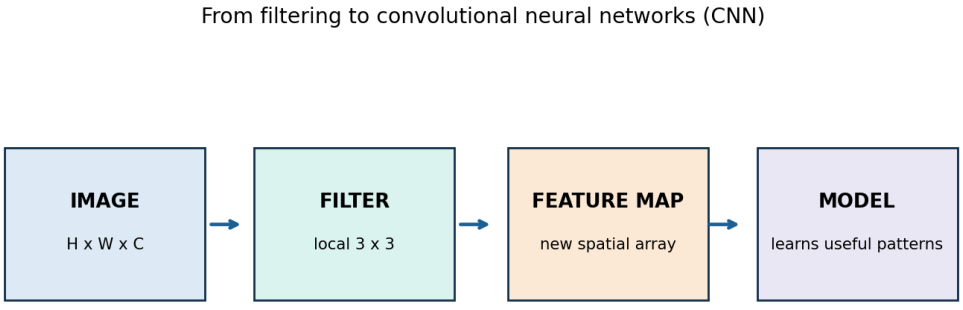
$$\text{image shape } (8,8) \;\rightarrow\; \text{flatten} \;\rightarrow\; \text{feature vector length } 64$$

```
from sklearn.datasets import load_digits
digits = load_digits()
X = digits.images.reshape(len(digits.images), -1)
y = digits.target
print(digits.images.shape)   # (n, 8, 8)
print(X.shape)               # (n, 64)
```

הספרייה load_digits מספקת דוגמאות אמיתיות של ספרות בכתב יד ברזולוציה של 8×8, עם ערכי עוצמה בין 0 ל־16. כדי להציגן בטווח 0–255 אפשר להכפיל ב־255/16, אבל זאת המרה לייצוג תצוגה; אין לבלבל בינה לבין העובדה שהערכים המקוריים של המאגר אינם שמונה־ביט מלאים.

בדיקת הבנה: מדוע תמונה של ספרה מתאימה גם ל־KNN וגם ל־CNN, אך לרשת CNN יש יתרון מבני בעבודה על שכונות פיקסלים?

16. גשר לפרק CNN: מסנן לומד לעומת מסנן קבוע



עד כה בחרנו ביד את משקולות המסננים: למשל כל משקל 1/9 לצורך מיצוע או מקדמים קבועים להדגשת קצוות. ברשת קונבולוציה, לעומת זאת, שכבות הקונבולוציה מכילות משקולות שנלמדות מן הנתונים בזמן האימון. בשכבות הראשונות עשויים להופיע מסננים המגיבים לקצוות או למרקמים; בשכבות עמוקות יותר יכולים להיווצר ייצוגים מורכבים. זהו תיאור רעיוני, לא הבטחה שכל רשת תלמד בדיוק אותו סדר של תכונות.

דימוי מועיל הוא ההבדל בין מסננת קבועה לבין כלי שמכוונים את המשקולות שלו לפי טעויות החיזוי. כעת אנו לומדים את הפעולה על חלון קטן; בפרק הרשתות נלמד כיצד השגיאה משנה את המשקולות בעזרת תהליך אימון.

למידה עמוקה אינה מחליפה חשיבה על איכות הנתונים. גם רשת עוצמתית עלולה להיכשל אם התמונות אינן מייצגות מצבי תאורה שונים, אם יש טעויות בתוויות, או אם נתוני Test הושפעו מן התהליך שבו בנינו את המודל.

17. הנדסת ניסוי נכון — מה בודקים לפני שמשווים מודלים?

נניח שמודל A אומן על תמונות בהירות ומודל B על תמונות כהות שנוצרו באמצעות עיבוד שונה. אם גם הנתונים, גם סוג העיבוד וגם המודל השתנו במקביל, קשה לדעת מה גרם לשינוי בביצועים. לכן ניסוי הוגן משנה גורם אחד בכל פעם, משאיר את נתוני הבדיקה מופרדים ומדווח לא רק על ציון אחד אלא גם על סוגי הטעויות.

לצורך פרויקט גמר רצוי לתעד תהליך קבוע: מקור התמונות והרשאות השימוש, גודל הקלט, סדר ערוצי הצבע, המרה לאפור או שמירת RGB, שיטת Resize, טווח נרמול והדרך שבה חולקו הדוגמאות ל־Train ול־Test. אם אותה תמונת מקור צולמה כמה פעמים בזוויות דומות, כדאי לבדוק שלא פיזרנו צילומים כמעט זהים בין אימון לבדיקה ויצרנו הערכה אופטימית מדי.

תיעוד מוצע	שאלת בדיקה	שלב
מקור, רישיון, סיכונים	האם התמונות דומות למצבים בעולם האמיתי?	נתונים
H×W×C, RGB/gray	מהם גודל התמונה וסדר הערוצים?	קלט
Crop, Resize, Normalize	האם שינינו את המידע?	עיבוד
חלוקה, מקור, מניעת כפילות	האם נשמר Test עצמאי?	בדיקה
מטריצת בלבול ודוגמאות	היכן המודל נכשל?	תוצאה

18. התנסות מודרכת מלאה — תמונה, מערך וגרף

הקוד הבא יוצר תמונה קטנה מקורית, מציג את צורת המערך ומחשב תמונה אפורה. הוא אינו דורש הורדת קובץ חיצוני, ולכן ניתן להפעילו במחברת Colab חדשה. לפני ההרצה נסו לנבא מה יהיו ממדי המערכים.

```
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image

rgb = np.zeros((12, 16, 3), dtype=np.uint8)
rgb[:] = [25, 45, 90]
rgb[2:10, 3:13] = [245, 185, 55]
rgb[4:8, 5:11] = [25, 160, 160]

r, g, b = [rgb[:, :, i].astype(float) for i in range(3)]
gray = np.clip(0.299*r + 0.587*g + 0.114*b,
               0, 255).astype(np.uint8)
```

```
print('RGB shape:', rgb.shape)
print('Gray shape:', gray.shape)
fig, ax = plt.subplots(1, 2, figsize=(7, 3))
ax[0].imshow(rgb)
ax[1].imshow(gray, cmap='gray', vmin=0, vmax=255)
for a in ax: a.axis('off')
plt.show()
```

לאחר ההרצה הסבירו איזו פעולה שינתה את מספר הערוצים, ואיזו פעולה רק הציגה אותם על המסך. את התשובה נסחו בעברית ולא רק באמצעות הדפסת הצורות.

19. משימות תרגול מדורגות

רמה א׳ — הבנת ייצוג ופעולות

1. תמונה אפורה בגובה 20 ורוחב 30: כמה פיקסלים יש בה? מהי צורת המערך? מה יהיה ההבדל אם היא תמונת RGB?

2. הציגו שלושה פיקסלים שערכיהם 0, 128, 255 והסבירו כיצד כל אחד ייראה בתמונה אפורה.

3. עבור הפיקסל (200,100,50), בצעו חישוב ידני של ההמרה לגווני אפור. הסבירו מדוע הממוצע הפשוט נותן תוצאה אחרת.

4. כתבו את טווח השורות והעמודות בביטוי image[1:4, 2:6]. כמה תאים ייבחרו?

5. קבעו אם שינוי colormap בעת plt.imshow משנה את נתוני התמונה או רק את התצוגה.

רמה ב׳ — חשיבה מתמטית וניתוח קוד

6. לפיקסלים 30, 210, 250 מוסיפים β=40. חשבו את הערכים לפני clip ואחריו והסבירו מדוע דרושה פעולת clipping.

7. קבלו חלון 3×3 שבו במרכז 90 וכל שמונת השכנים הם 10. מצאו ממוצע וחציון, והשוו מה כל מסנן יעשה לערך המרכזי.

8. תמונה בגודל 10×12 עוברת מסנן 3×3, ללא Padding וב־Stride של 1. מה יהיו ממדי הפלט? נמקו.

9. השוו בין Resize בשיטת Nearest Neighbor לבין Bilinear. איזה מידע לא ניתן לשחזר מתוך תמונה קטנה במיוחד?

10. התלמיד טוען ש־Normalization ו־Thresholding הן אותה פעולה. כתבו דוגמה מספרית הסותרת את הטענה.

רמה ג׳ — העברה לבגרות ולפרויקט גמר

11. מערכת לזיהוי ספרות מצליחה באימון ונכשלת בצילומים בתאורה חלשה. הציעו ניסוי מבוקר שיבדוק את השפעת עיבוד הבהירות בלי לשנות את נתוני Test לצורך בחירת הפרמטרים.

12. שתי קבוצות תמונות הן בעלות היסטוגרמת בהירות זהה, אך שונות במיקום קווי המתאר. הסבירו מדוע לא כדאי לבסס סיווג ספרות על היסטוגרמה בלבד.

13. הציעו מדוע רשת CNN שמקבלת תמונה דו־ממדית עשויה להיות עדיפה על מסווג רגיל המקבל וקטור שטוח, גם כשמספר ערכי הקלט זהה.

14. תארו שלושה פרטים שמוכרחים להופיע בתיעוד של שלב עיבוד התמונה בפרויקט גמר, ונמקו את חשיבותם.

15. תכננו בדיקת שגיאות למסווג תמונות: אילו מקרים תציגו בפני הבוחן כדי להוכיח שאתם מבינים גם את מגבלות המערכת?

20. שאלת הערכה מקורית בסגנון בגרות

מערכת למיון מוצרים מצלמת כרטיס שעליו מופיע מספר סידורי. כל כרטיס נחתך לתמונה אפורה בגודל 6×6 פיקסלים, ומסווג באמצעות מודל שנלמד מנתוני אימון מסומנים. נתונה המטריצה הבאה המתארת חלון מקומי:

10	20	10
20	100	20
10	20	10

א. הסבירו מהו ערך הפיקסל המרכזי ומהו טווח הערכים האפשרי בתמונה מטיפוס uint8. ב. חשבו את ערך המסנן הממוצע על החלון והראו את שלבי הסכימה. ג. הסבירו כיצד סינון עשוי גם לפגוע בזיהוי הספרה. ד. אם נבצע חיתוך תמונה ו־Resize, באיזה סדר הייתם פועלים ומדוע? ה. הסבירו מדוע צריך לשמור קבוצת Test נפרדת בעת בחירת שיטת עיבוד תמונה.

דרישת ההערכה: נימוק מלא במילים, חישוב מנומק והבחנה בין פעולת עיבוד קבועה לבין מודל שלומד מנתונים. זו שאלה מקורית שנכתבה לצורכי ההוראה, ולא שאלה מקורית ממאגר בחינות משרד החינוך.

21. מעבדת Google Colab ומדדי הצלחה

המעבדה המצורפת כוללת 12 משימות המתקדמות מזיהוי ממדים וערכי פיקסלים אל יצירת מסננים וניתוח ניסוי. בתאי התרגול מופיעים מקומות להשלמת קוד, ולא פתרונות מלאים. לפני כל תא יש לכתוב תחזית לגבי התוצאה; אחרי ההרצה יש להסביר אותה. מעבדה תיחשב מוצלחת רק אם התלמיד מסוגל להסביר את צורות המערכים, את שינויי הערכים ואת הסיכון לאיבוד מידע.

מה התלמיד מציג	יכולת
פירוש צורה (H,W) או (H,W,3)	ייצוג
המרה לאפור ומסנן ממוצע באופן ידני	חישוב
פעולות NumPy/Pillow מוסברות	קוד
השוואת שיטות עיבוד תוך שמירה על אותם נתונים	חקירה
הסבר מגבלה והשפעתה על AI	הגנה

22. הקשר בשלוש תכניות הלימוד

הסטטוס	תפקיד התוכן	תכנית
ידע קודם / גשר פדגוגי	חזרה על מערכים, מאפיינים, KNN והמחשה בסיסית	יחידה 3 — מדעי הנתונים
קישור מושגי; לא פרק תמונה נפרד	וקטורים, מטריצות, מרחקים, תקנון, חקר נתונים	יחידה 5 — נושאי ליבה
פרק 6: 15 שעות לפי מפת התכנית	עיבוד תמונה שיטתי, הכנת קלט וגשר ל־CNN	התמחות למידה עמוקה

הספר נשען על מפת ההתמחות השמורה במאגר GitHub שלנו. הגדרות, דוגמאות וניסויים נוספו כדי להנגיש לתלמידים רעיונות המופיעים בספרי יסוד כגון Hands-On Machine Learning, Mathematics for Machine Learning, Dive into Deep Learning ו־Grokking Deep Learning. כל הדוגמאות והאיורים בפרק נוצרו עבור החוברת; לא הועתקו מהם פרקים או איורים מוגנים.

23. הצעת חלוקת 15 שעות הוראה

תוצר	נושאים	שעות
קריאת מערכים ותיאור תמונה	מבוא, פיקסלים, ממדים, RGB ואפור	1–3
חישוב ידני ותרשים	אינדקסים, בהירות, ניגודיות והיסטוגרמה	4–6
השוואת עיבודים מודרכת	רעש, Crop, Resize, Normalize	7–9
מימוש מסנן וניתוח פלט	מסננים, קונבולוציה וזיהוי קצוות	10–12
הסבר פרויקט ומגבלות	הכנה ל־CNN, מעבדה, משימת הערכה	13–15

חלוקת השעות היא הצעה פדגוגית ל־15 שעות הסילבוס, ולא טבלה רשמית של משרד החינוך. ניתן להאריך או לקצר פעילויות בהתאם לרמת הכיתה, תוך שמירת תוצרי החובה שהמורה קובע על פי המסמך התקף.

מקורות והמשך קריאה

תכנית ההתמחות במאגר הקורס: https://github.com/samiabo75/Data-Science-ML-Course/tree/main/03-Deep-Learning-5-Units/06-Image-Processing

Aurélien Géron, Hands-On Machine Learning with Scikit-Learn and PyTorch — פרקי ייצוג והכנת נתונים ומודלי ראייה.

Zhang, Lipton, Li & Smola, Dive into Deep Learning — פרקי image processing ו־Convolutional Neural Networks; https://d2l.ai

Deisenroth, Faisal & Ong, Mathematics for Machine Learning — אלגברה ליניארית ופעולות מערכיות; https://mml-book.github.io

Pillow documentation: https://pillow.readthedocs.io

/NumPy documentation: https://numpy.org/doc/stable

scikit-learn load_digits: https://scikit-learn.org/stable/modules/generated/sklearn.datasets.load_digits.html

הערת מיפוי: הפרק מבוסס על כותרת פרק 6 והיקפו במאגר הסילבוס, אך ללא בדיקה חוזרת של כל שורות ה־PDF המקורי. ההרחבות בנושא קונבולוציה, הטיה וניהול ניסוי הוצגו כהכנה מקצועית ל־CNN ולפרויקט ולא כטענה אוטומטית לחובת בחינה.